Projektowanie i analiza algorytmów

Sprawozdanie

Algorytmy sortowania

Dimitrije Vlajsević, 284164

22.04.2026.

Spis treści

1 Wprowadzenie **3**

2 Opis algorytmów **3**

2.1 Quicksort 3

2.2 Introsort 3

2.3 Bucketsort 4

2.4 Drzewo binarne 4

3 Przebieg eksperymentów **5**

3.1 Wczytywanie danych 5

3.2 Sortowanie 5

3.3 Zapis wyników 5

4 Wyniki eksperymentów **6**

5 Analiza wyników **7**

5.1 Porównanie czasów 7

5.2 Zgodność z oczekiwaną złożonością 7

6 Podsumowanie i wnioski **8**

1 Wprowadzenie

Celem projektu było zaimplementowanie trzech algorytmów sortowania (Quicksort, Introsort oraz Bucketsort), przeprowadzenie eksperymentów na zbiorze danych filmowych pochodzącym z bazy IMDb, a także zapis posortowanych danych przy użyciu drzewa binarnego.

2 Opis algorytmów

2.1 Quicksort

Idea algorytmu polega na wyborze elementu zwanego *pivotem*, podziale tablicy na dwie części (elementy mniejsze i większe od pivota) oraz rekurencyjnym sortowaniu obu części [1].

W zaimplementowanej wersji stosowana jest strategia *median of three* [2]: pivot wybierany jest jako mediana pierwszego, środkowego i ostatniego elementu podtablicy, co pozwala uniknąć degeneracji do złożoności kwadratowej dla wielu typowych danych wejściowych (np. ciągów już posortowanych).

Złożoność obliczeniowa.

- Przypadek średni: $\Theta(n \log n)$
- Przypadek najgorszy: $\Theta(n^2)$ (np. dla ciągu już posortowanego przy złym wyborze pivota)
- Pamięć (stos rekurencji): $O(\log n)$ średnio, $O(n)$ najgorzej

2.2 Introsort

Introsort łączy zalety trzech algorytmów [3]:

1. **Quicksort** - efektywny w przypadku średnim;
2. **Heapsort** - gwarantuje złożoność $O(n \log n)$ w najgorszym przypadku; stosowany gdy głębokość rekurencji przekroczy próg $2\lfloor \log_2 n \rfloor$;
3. **Sortowanie przez wstawianie** - efektywne dla małych tablic.

Zaimplementowana wersja korzysta z podziału z *median of three* oraz granicy głębokości rekurencji wynoszącą $2\lfloor \log_2(p - l + 1) \rfloor$, gdzie l i p są odpowiednio lewym i prawym indeksem sortowanego przedziału.

Złożoność obliczeniowa.

- Przypadek średni: $\Theta(n \log n)$
- Przypadek najgorszy: $\Theta(n \log n)$ (dzięki przełączeniu na heapsort)
- Pamięć: $O(\log n)$

2.3 Bucketsort

Dane są rozdzielane do k kubełków o równej szerokości zakresu, każdy kubełek jest sortowany niezależnie (tutaj: sortowaniem przez wstawianie), a następnie kubełki są łączone w wynikową tablicę [4].

W zaimplementowanej wersji liczba kubełków wynosi $k = \lfloor \sqrt{n} \rfloor + 1$, gdzie n to liczba elementów. Zakres każdego kubełka wyznaczono jako $(\max - \min)/(k - 1)$.

Złożoność obliczeniowa.

- Przypadek średni: $\Theta(n + k)$ dla jednostajnie rozłożonych danych (gdzie k to liczba kubełków); dla $k = \Theta(n)$ daje $\Theta(n)$;
- Przypadek najgorszy: $\Theta(n^2)$ - gdy wszystkie elementy trafiają do jednego kubełka;
- Pamięć: $O(n + k)$

Bucketsort jest algorytmem *nieliniowym porównawczo* - jego efektywność zależy od rozkładu danych. Dla ocen z bazy IMDb, które są wartościami zmiennoprzecinkowymi z zakresu $[1.0, 10.0]$, rozkład jest stosunkowo równomierny, co sprzyja działaniu algorytmu.

2.4 Drzewo binarne

Do efektywnego zapisu posortowanych danych zastosowano czerwono-czarne drzewo binarne [5], czyli takie w którym:

1. Każdy węzeł jest czerwony lub czarny.
2. Korzeń jest czarny.
3. Liście są czarne.
4. Jeśli węzeł jest czerwony, obaj jego synowie są czarni (analogicznie w drugą stronę).
5. Dla każdego węzła wszystkie proste ścieżki do liści zawierają tę samą liczbę czarnych węzłów.

Dzięki tym właściwościom wysokość drzewa jest zawsze $O(\log n)$, co gwarantuje czas wstawiania i wyszukiwania rzędu $O(\log n)$. Przejście po drzewie daje elementy w kolejności niemalejącej.

3 Przebieg eksperymentów

3.1 Wczytywanie danych

Pliki wejściowe wczytywane są w dwóch etapach:

1. **Tytuły filmów** (`title.basics.tsv`): plik zawiera ok. 9,7 mln rekordów. Aby skrócić czas wczytywania, zastosowano następujące optymalizacje: odczyt całego pliku do bufora (`rdbuf`) zamiast wielokrotnego `getline`, parsowanie bezpośrednio po wskaźniku `const char*` zamiast `istringstream`, rezerwacja pamięci dla `unordered_map` z niskim współczynnikiem wypełnienia (`max_load_factor(0.25f)`). Czas wczytywania: **29 734 ms**.
2. **Oceny** (`title.ratings.tsv`): plik zawierał 1,295 mln rekordów. Puste lub nieprawidłowe wartości oceny zostały pominięte. Czas wczytywania: **10 645 ms**.

3.2 Sortowanie

Każdy algorytm wywoływany był na kopii danych (niezależnie dla każdego rozmiaru), by nie wpływać na kolejne pomiary.

3.3 Zapis wyników

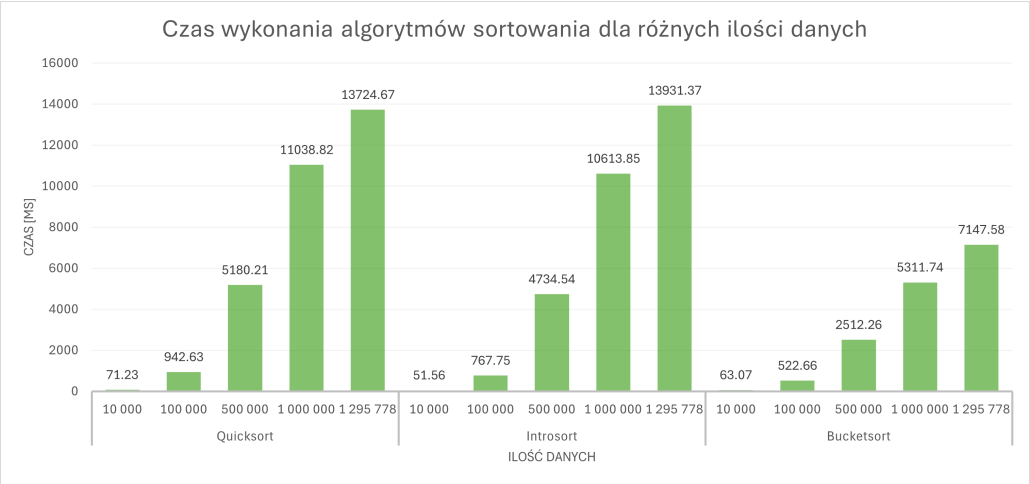
Czas zapisu posortowanych danych do pliku za pomocą drzewa wynosił **4 103 ms**.

4 Wyniki eksperymentów

Algorytm	N	Czas [ms]	Średnia	Mediana
Quicksort	10 000	71.23	5.99	6.10
	100 000	942.63	6.18	6.30
	500 000	5180.21	6.76	6.90
	1 000 000	11038.82	6.90	7.10
	1 295 778	13724.67	6.95	7.10
Introsort	10 000	51.56	5.99	6.10
	100 000	767.75	6.18	6.30
	500 000	4734.54	6.76	6.90
	1 000 000	10613.85	6.90	7.10
	1 295 778	13931.37	6.95	7.10
Bucketsort	10 000	63.07	5.99	6.10
	100 000	522.66	6.18	6.30
	500 000	2512.26	6.76	6.90
	1 000 000	5311.74	6.90	7.10
	1 295 778	7147.58	6.95	7.10

Tabela 1: Czasy sortowania, średnia i mediana oceny dla badanych algorytmów.

Wszystkie trzy algorytmy zwróciły identyczne wartości średniej i mediany dla tych samych podzbiorów danych - potwierdza to poprawność implementacji.



Rysunek 1: Porównanie czasów sortowania

5 Analiza wyników

5.1 Porównanie czasów

Na podstawie tabeli 1 można zaobserwować:

- **Bucketsort** jest wyraźnie najszybszy dla dużych danych: dla $n = 1\,295\,778$ osiąga czas 7 148 ms, czyli niemal o połowę mniej niż Quicksort (13 725 ms) i Introsort (13 931 ms).
- **Introsort** jest nieznacznie szybszy od Quicksort dla małych i średnich n (10 000 - 500 000), co jest zgodne z teorią (szybszy heapsort dla niekorzystnych permutacji, insertion sort dla małych podtablic). Dla największego zestawu Introsort okazał się nieznacznie wolniejszy od Quicksorta - prawdopodobnie ze względu na zwiększoną liczbę przełączeń do heapsorta dla konkretnych danych.
- Czasy Quicksort i Introsort rosną w przybliżeniu liniowo względem $n \log n$, co jest zgodne z ich oczekiwaną złożonością.
- Czasy Bucketsort rosną wolniej, co wskazuje na zbliżone do liniowego zachowanie wynikające z równomiernego rozkładu danych.

5.2 Zgodność z oczekiwaną złożonością

Dla algorytmów opartych na porównaniach (Quicksort, Introsort) oczekujemy złożoności $\Theta(n \log n)$. Stosunek czasów dla kolejnych rozmiarów danych pozwala zweryfikować tę hipotezę:

N_1	$N_2 = 10N_1$	$\frac{T(N_2)}{T(N_1)}$	$\frac{N_2 \log N_2}{N_1 \log N_1}$
10 000	100 000	13.23	13.25
100 000	1 000 000	11.71	11.67

Tabela 2: Weryfikacja złożoności Quicksort: stosunek $T(10N)/T(N)$ i teoretyczna wartość $(10N \log 10N)/(N \log N)$.

Uzyskane stosunki są bardzo bliskie wartościom teoretycznym, co potwierdza, że Quicksort zachowuje się zgodnie z $\Theta(n \log n)$.

Dla Bucketsort oczekujemy złożoności zbliżonej do liniowej. Stosunek $T(1\,000\,000)/T(100\,000) \approx$ 10,2, co jest zgodne z $O(n)$ (stosunek wynosi 10). Potwierdza to również, że dane z bazy IMDb są dostatecznie równomiernie rozłożone.

6 Podsumowanie i wnioski

W ramach projektu zaimplementowano i przetestowano trzy algorytmy sortowania - Quicksort, Introsort oraz Bucketsort - na zbiorze danych filmowych z bazy IMDb liczącym do 1,3 mln rekordów. Wyniki eksperymentów prowadzą do następujących wniosków:

1. Wszystkie trzy algorytmy poprawnie sortują dane; identyczne wartości statystyk (średnia, mediana) potwierdzają brak błędów implementacji.
2. **Bucketsort** okazał się najszybszy dla wszystkich rozmiarów zbioru. Jego przewaga rośnie wraz z n, co wynika z liniowej złożoności przy równomiernym rozkładzie ocen. W zbiorze IMDb oceny skupiają się w przedziale 5,0-9,0, co sprzyja równomiernemu zapełnieniu kubełków.
3. **Introsort** jest nieznacznie efektywniejszy niż Quicksort dla małych i średnich zbiorów, eliminując ryzyko zachowania kwadratowego dzięki przełączeniu na heapsort.
4. Zmierzone czasy sortowania Quicksort i Introsort są zgodne z teoretyczną złożonością $\Theta(n \log n)$, a Bucketsort zachowuje się jak $O(n)$ dla badanego zbioru.
5. Drzewo czerwono-czarne umożliwiło efektywny, posortowany zapis danych do pliku wyjściowego w czasie $O(n \log n)$.

Listing 1: Początkowy fragment pliku wyjściowego - filmy posortowane rosnąco wg. ocen

```
1       Graustark
1.0     Young Lochinvar
1.0     Wild Justice
1.0     Back to Liberty
1.0     Aguilas de acero o los misterios de Tanger
1.0     The Rip-Tide
1.0     Through Fire and Water
1.0     Steckler Interviews
1.0     Shadows
1.0     The Reel Deal
1.0     Big News
1.0     Passport to the World
1.0     Target World
1.0     The Case He Couldn't Crack
1.0     Reality
1.0     QIK2JDG
1.0     Folkets egen dronning
1.0     Weavers of Fortune
1.0     DreamGirls: Real Adventures 37
```

Literatura

[1] GeeksforGeeks, "QuickSort - Data Structure and Algorithm Tutorials," dostępne online: https://www.geeksforgeeks.org/quick-sort-algorithm/

[2] GeeksforGeeks, "Median of Three Partitioning in QuickSort," dostępne online: https://www.geeksforgeeks.org/median-of-three-partitioning-quicksort/

[3] Wikipedia, "Introsort," dostępne online: https://en.wikipedia.org/wiki/Introsort

[4] GeeksforGeeks, "Bucket Sort," dostępne online: https://www.geeksforgeeks.org/bucket-sort-2/

[5] Wikipedia, "Red-black tree," dostępne online: https://en.wikipedia.org/wiki/Red-black_tree

[6] IMDb, "IMDb Non-Commercial Datasets," dostępne online: https://developer.imdb.com/non-commercial-datasets/